WORKPULSE

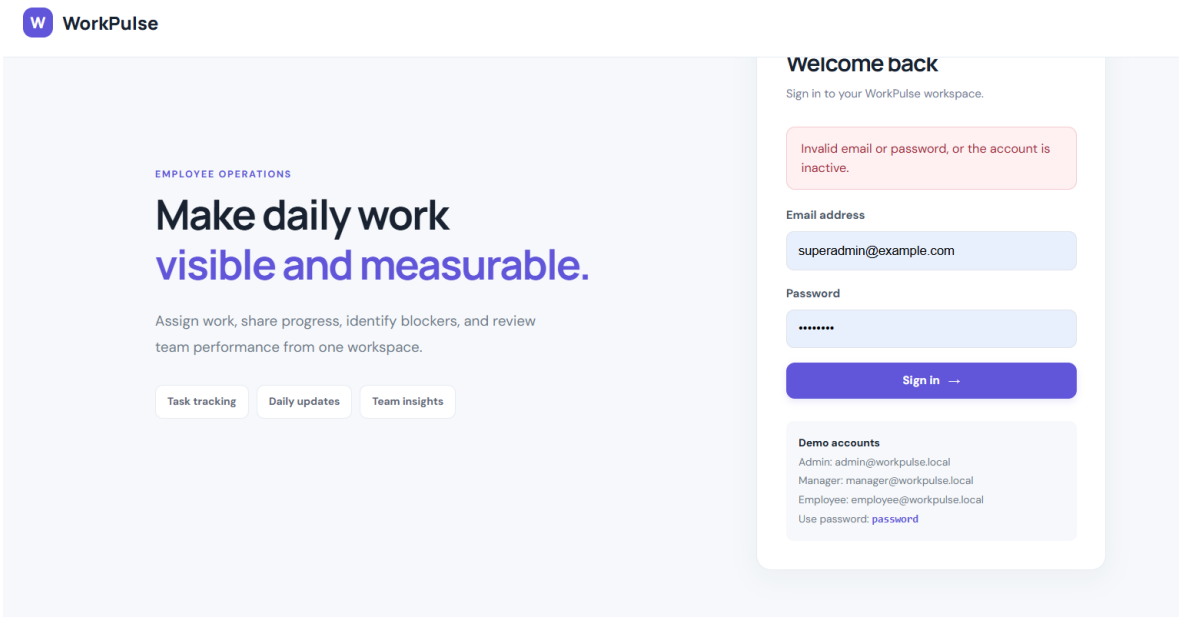
Employee Daily Work Update & Performance Evaluation System

Application documentation • Workflow • Technology stack • Database design • Setup guide

Item	Description
Project type	PHP + MySQL web application
Purpose	Assign work, capture daily progress, monitor deadlines, and support manager evaluations
Intended audience	Recruiter, technical evaluator, developer, and project reviewer
Local environment	XAMPP on Windows (Apache + PHP + MySQL)

Project summary
WorkPulse is a role-based work management application designed to make employee tasks, daily progress, blockers, deadlines, and review feedback visible in one place. It uses native PHP with PDO and a MySQL relational database, with a responsive HTML/CSS/JavaScript interface.

Application screen (local test)



Prepared for technical assignment review. This document describes the current implementation and distinguishes implemented features from production recommendations.

1. Application overview

The application supports three roles and five main operational areas: task management, daily work updates, dashboard metrics, reports, and performance evaluations. Admin users can also create manager and employee accounts.

Objectives

• Give managers a structured way to allocate tasks and set due dates and priorities.

• Allow employees to report completed work, next steps, pending items, and blockers.

• Identify overdue work and surface team progress on a dashboard.

• Provide simple reports and export employee metrics to CSV.

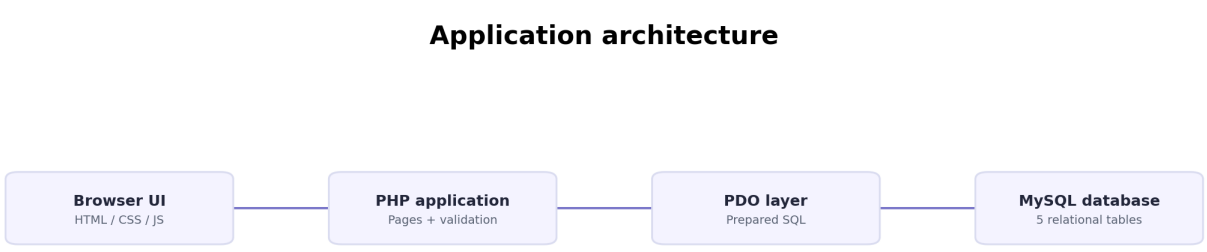
• Record structured manager evaluations with ratings and written feedback.

2. Technology stack

Technology	Use in this project
PHP (native/Core PHP)	Server-side pages, request handling, session authentication, validation, role checks, and business logic.
PDO (PHP Data Objects)	Database access using prepared statements and exception-based error handling.
MySQL / InnoDB	Relational storage, foreign keys, indexes, constraints, and seeded demo records.
HTML5	Forms, tables, navigation, semantic page structure.
CSS3	Responsive layout, cards, status labels, forms, and visual styling in assets/style.css.
JavaScript	Small client-side interactions, including textarea resizing and confirmation hooks.
Apache via XAMPP	Local web server used to run PHP pages.
phpMyAdmin	Local database import and inspection tool; not part of the app runtime.

Not used in this build: Laravel, Composer, React, Bootstrap, jQuery, or a frontend build tool. The application is intentionally implemented with native PHP and plain HTML/CSS/JavaScript.

3. Architecture



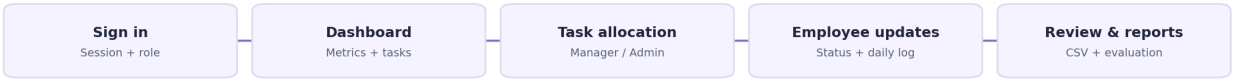
Typical request flow

• The browser requests a PHP page such as login.php, tasks.php, or reports.php.

- PHP loads config.php and shared helpers from includes/functions.php.
- Authentication and role checks run before protected data is shown.
- PDO executes parameterized SQL against MySQL.
- The PHP page escapes displayed values and returns HTML styled by assets/style.css.

4. End-to-end workflow

WorkPulse — End-to-end application workflow

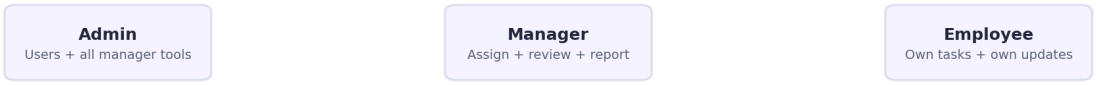


Step-by-step workflow

Step	Actor	Workflow
1. Sign in	All roles	User enters email and password. Password is checked with password_verify(); a session is created and the session ID is regenerated.
2. Dashboard	All roles	The dashboard shows role-appropriate task counts, completion rate, pending work, overdue work, and recent updates.
3. Allocate task	Admin / Manager	Enter title, description, employee, due date, and priority. The task is stored with its assigner and default status To-do.
4. Work on task	Employee	View assigned tasks, open details, and change status to In progress, Blocked, or Completed.
5. Submit daily update	Employee	Record work date, completed work, next steps, pending tasks, and blockers. The update appears in history.
6. Review team activity	Admin / Manager	Review team tasks and daily updates; overdue tasks are highlighted when due date is before today and status is not completed.
7. Reporting	Admin / Manager	View employee-level assigned/completed/overdue task metrics and daily-update counts; export the report as CSV.
8. Evaluation	Admin / Manager	Score quality, communication, ownership, and timeliness from 1–5, add comments, and save the average score.

5. Role permissions

Role-based access model



Capability	Admin	Manager	Employee
View dashboard	Yes	Yes	Own scope
Create and assign tasks	Yes	Yes	No
Update task status	Yes	Yes	Assigned tasks
View daily updates	Team	Team	Own updates
View reports / CSV export	Yes	Yes	No
Create performance evaluations	Yes	Yes	No
Create user accounts	Yes	No	No
Delete tasks	Yes	No	No

6. Feature documentation

6.1 Dashboard

• Displays total tasks, completed tasks, pending/in-progress/blocked tasks, and overdue tasks.

• Shows completion rate as completed tasks divided by total tasks in the relevant scope.

• Displays a task overview and recent daily updates.

• Shows a daily check-in prompt based on whether the current user submitted an update today.

6.2 Task management

• Managers can create tasks and assign them to active employee accounts.

• Each task stores title, description, assignee, assigner, due date, priority, status, and timestamps.

• Supported statuses: To-do, In progress, Blocked, Completed.

• The task list can be filtered by status or overdue-only state.

• Employees can only view and update tasks assigned to them; manager/admin views are broader.

6.3 Daily work updates

• An update includes work date, completed work, to-do/next steps, pending work, and blockers/support needed.

• At least one content field must be supplied before submission.

• Users can only see their own update history; managers/admins can see team updates.

• The app records the creation timestamp and a basic activity log entry.

6.4 Reports and CSV export

• The report shows employee, email, assigned tasks, completed tasks, completion rate, overdue tasks, update count, and updates containing blockers.

• Date filters apply to daily-update counts. Task totals are based on current task status, not historical status as of the selected period.

• CSV export is available for the report so data can be reviewed in spreadsheet software.

6.5 Performance evaluation

• Managers rate quality, communication, ownership/initiative, and timeliness on a 1–5 scale.

• The overall score is the arithmetic mean of the four dimension scores.

• Each evaluation records the employee, reviewer, review period, ratings, comments, and creation timestamp.

• Scores are intended to support a manager conversation and should not be treated as a fully automated employment decision.

7. Database design

Table	Purpose / key fields
users	User identity, email, password_hash, role, active flag, timestamps. Email is unique.
tasks	Task content, assigned_to, assigned_by, due_date, priority, status, timestamps. Foreign keys link to users.
daily_updates	User, work date, completed_work, todo_work, pending_work, blockers, created_at.
evaluations	Employee/reviewer, review period, four rating fields, overall_score, comments.

Table	Purpose / key fields
activity_logs	Actor user, action, details, timestamp for basic auditability.

Relationships

One user can have many assigned tasks, daily updates, and evaluations as an employee or reviewer. Each task references an assignee and the user who assigned it. Daily updates reference their author. Evaluation records reference both the employee and reviewer. Foreign keys help preserve referential integrity.

8. Security and validation

Control	Implementation / limitation
Password handling	Passwords are stored as hashes and checked using password_verify(). Demo accounts use a shared demo password and must be changed before deployment.
SQL injection mitigation	PDO prepared statements are used for user-provided values in database queries.
CSRF protection	State-changing forms include a session CSRF token and verify it on POST.
XSS mitigation	Displayed user-controlled values are escaped through the e() helper using htmlspecialchars().
Authentication	Protected pages require a logged-in session; session ID is regenerated after successful login.
Authorization	Role checks and task ownership conditions limit access to sensitive operations.
Production hardening	HTTPS, secure cookie settings, rate limiting, password reset/change, least-privilege DB account, backup/retention policy, and automated tests remain recommended.

9. Local installation on XAMPP

1. Extract employee_work_performance_system.zip.

2. Copy the folder to C:\\xampp\\htdocs\\employee_work_performance_system.

3. Start Apache and MySQL from the XAMPP Control Panel.

4. Open phpMyAdmin at http://localhost/phpmyadmin/ and import database.sql.

5. Open config.php and set database host, database name, username, and password for the local MySQL instance.

6. Set BASE_URL to /employee_work_performance_system when the project is served from that folder.

7. Visit http://localhost/employee_work_performance_system/login.php.

Demo accounts

Role	Email	Password
Admin	admin@workpulse.local	password
Manager	manager@workpulse.local	password
Employee	employee@workpulse.local	password
Employee	asha@workpulse.local	password

Important: Demo credentials are only for local evaluation. Do not expose this installation publicly with the demo passwords or root database credentials.

10. Suggested test checklist

- ■ Sign in as Admin, Manager, and Employee; verify each role sees the expected navigation.
- ■ As Manager, create a task with an employee, due date, and priority.
- ■ As Employee, open the assigned task and change its status.
- ■ Submit a daily update with completed work, pending work, next steps, and a blocker.
- ■ Confirm employees cannot open another employee's task/update by changing IDs in URLs.

- ■ As Manager, confirm the dashboard and reports reflect the task and update.
- ■ Export the report to CSV and open it in a spreadsheet application.
- ■ Create an evaluation and verify the overall score is the average of the four ratings.
- ■ Try empty/invalid form values and verify helpful validation errors.

11. Improvements proposed beyond core requirements

Notifications and reminders: Email/in-app reminders for tasks approaching deadlines, overdue tasks, and missing daily updates.

Task comments and attachments: Conversation thread and supporting files per task, with file type/size validation.

Task history: Record each status transition with old value, new value, actor, and timestamp for historical reporting.

Team scoping: Restrict each manager to their own team or department rather than all employees.

Employee self-service: Add password change/reset and profile management.

More robust analytics: Weekly/monthly trends, workload distribution, average time-to-completion, and update punctuality.

Automated tests: Unit/integration tests for login, role permissions, task ownership, reports, and evaluation calculations.

Production readiness: Secure session cookies, HTTPS, rate limiting, secrets outside source control, database backups, logging/monitoring, and retention rules.

12. Known scope notes

• This is an assignment/demo application, not a fully hardened production HR platform.

• The report's task metrics use current task states; no historical task status snapshot table is implemented.

• Daily updates can be submitted more than once per day; the app displays update history rather than enforcing one update per employee per day.

• The activity log is written by core actions, but a dedicated audit-log viewer and tamper-resistant audit storage are not included.

• No email notification service, file attachments, password-reset workflow, or automated test suite is included in the current build.

Closing summary

WorkPulse demonstrates a practical PHP/MySQL workflow for task allocation, employee work reporting, manager visibility, CSV reporting, and structured performance feedback. The implementation emphasizes readable native PHP, relational data integrity, role-aware access, and baseline web security practices, while documenting the next steps needed for production readiness.